

Lab Assignment: Buffer, Stack, and Heap Overflows

Course: Systems Security / Low-Level Programming

Topic: Memory Corruption Vulnerabilities

Part 1: Theoretical Background

Before attempting the exercises, read the following concepts to understand the mechanics of memory corruption.

1. What is a Buffer Overflow?

A buffer overflow occurs when a program writes more data to a buffer (a temporary storage area) than it can hold. Because the program does not verify the length of the data being written, the extra data overwrites adjacent memory locations.

2. Stack Overflow

The **Stack** is a region of memory that stores temporary variables created by each function (local variables) and context for function calls (return addresses).

- **Mechanism:** When a function is called, a "Stack Frame" is pushed. This contains the return address (where the CPU should go after the function finishes) and local variables.
- **The Vulnerability:** If a local buffer (array) is overflowed, the data grows upwards (or downwards, depending on architecture) towards the return address.
- **The Exploit:** An attacker can overwrite the **Return Address** with a specific address of their choosing (e.g., a "Win" function or shellcode), hijacking the control flow of the program.

3. Heap Overflow

The **Heap** is a region of memory used for dynamic memory allocation (e.g., using `malloc()` in C or `new` in C++).

- **Mechanism:** Data on the heap is stored in "chunks." These chunks often contain headers (metadata) telling the computer how big the chunk is and whether it is free.
- **The Vulnerability:** If a program writes past the end of a dynamically allocated object, it might overwrite the data of the *next* object in the heap, or overwrite the metadata of the heap manager itself.
- **The Exploit:** This is often used to manipulate function pointers stored in heap objects or to corrupt critical data flags (like `isAdmin = true`).

Part 2: Environment Setup

To perform these exercises, you must compile the code without modern security protections (like Stack Canaries or ASLR).

System Requirements: Linux Environment (Ubuntu VM or WSL recommended).

Compiler: GCC

Standard Compilation Flags for this Lab:

When compiling the exercises below, use the following flags to disable protections:

```
gcc -fno-stack-protector -z execstack -no-pie -o output_name source_file.c
```

- `-fno-stack-protector`: Disables the "canary" value that detects buffer overflows.
- `-z execstack`: Allows code execution on the stack.
- `-no-pie`: Disables Position Independent Executable (keeps memory addresses static).

Part 3: Practical Exercises

Exercise 1: The Stack Overflow (Variable Corruption)

Objective: Overwrite a local variable to change program logic without changing the code.

Code (stack_var.c):

```
#include <stdio.h>

#include <string.h>
#include <stdlib.h>

int main(int argc, char *argv[]) {
    int secret_pass = 0;
    char buffer[16];

    if (argc < 2) {
        printf("Usage: %s <input_string>\n", argv[0]);
        return 1;
    }

    // VULNERABILITY: strcpy does not check bounds
    strcpy(buffer, argv[1]);

    printf("Buffer content: %s\n", buffer);
    printf("Secret Pass Value: %d\n", secret_pass);

    if (secret_pass != 0) {
        printf("SUCCESS! You have corrupted the variable and bypassed the check!\n");
    } else {
        printf("Failed. Secret pass is still 0.\n");
    }

    return 0;
}
```

Lab Task:

1. Compile the code.
2. Run the program with a short string: `./stack_var hello`
3. Run the program with a string longer than 16 characters.
4. Determine exactly how many characters are needed to overwrite `secret_pass`.

Exercise 2: Stack Overflow (Control Flow Hijacking)

Objective: Redirect the program execution to a function that is never supposed to be called (`secret_function`).

Code (stack_control.c):

```
#include <stdio.h>
#include <string.h>
#include <stdlib.h>

void secret_function() {
    printf("\n[!!!] YOU BROKE IN! Secret Function Executed! [!!!]\n");
    exit(0);
}

void input_handler() {
    char buffer[30];
    printf("Enter some text: ");
    // VULNERABILITY: gets() is highly dangerous
    gets(buffer);
    printf("You entered: %s\n", buffer);
}

int main() {
    input_handler();
    printf("Normal execution completed.\n");
    return 0;
}
```

Lab Task:

1. Compile the code.
2. Use `objdump` or `gdb` to find the memory address of `secret_function`.
 - Command: `objdump -d stack_control | grep secret_function`
3. Calculate the padding needed to reach the Return Address on the stack.
4. Construct a payload (using Python or Perl) to fill the buffer, overwrite the saved base pointer, and inject the address of `secret_function` into the return address slot.
 - *Hint:* Addresses must be in Little Endian format (reversed).

Exercise 3: Heap Overflow (Data Corruption)

Objective: Overwrite dynamic memory data to escalate privileges.

Code (heap_overflow.c):

```
#include <stdio.h>
#include <string.h>
#include <stdlib.h>
typedef struct {
    char name[20];
    int priority;
} UserRequest;

int main(int argc, char *argv[]) {
    if (argc < 3) {
        printf("Usage: %s <name> <description>\n", argv[0]);
        return 1;
    }

    UserRequest *req = malloc(sizeof(UserRequest));
    char *description = malloc(20);

    // Initialize Request
    req->priority = 1; // 1 = Low priority, 99 = Admin priority
    strcpy(req->name, "Guest");

    // VULNERABILITY: Copying argv[1] into description without checking length.
    // In many heap implementations, 'req' might be allocated directly AFTER 'description'.
    strcpy(description, argv[1]);

    // Copying argv[2] to name (safe copy for this example)
    strncpy(req->name, argv[2], 19);

    printf("Description: %s\n", description);
    printf("User: %s\n", req->name);
    printf("Priority Level: %d\n", req->priority);

    if (req->priority == 99) {
        printf(">> ADMIN ACCESS GRANTED <<\n");
    }
    free(description);
    free(req);
    return 0;
}
```

Lab Task:

1. Compile the code.
2. Understand that in the Heap, description and req are likely allocated next to each other.
3. Provide an input for <name> (which goes into description) that is long enough to overflow out of the description chunk and overwrite the priority integer in the req chunk.
4. Try to set the priority specifically to 99.

Part 4: Lab Questions & Report

Submit a report answering the following:

Q1: Analysis of Exercise 1

- Why did the secret_pass variable change value even though you never assigned a value to it in the code?
- Draw a rough diagram of what the stack memory looked like before and after the strcpy operation.

Q2: Analysis of Exercise 2

- What is the "Return Address" and why is it stored on the stack?
- Why is gets() considered a banned function in modern C programming? What function should replace it?

Q3: Analysis of Exercise 3

- How does the Heap differ from the Stack regarding memory growth?
- If you successfully overwrote the priority, explain how you calculated the specific character input required to generate the integer 99.

Q4: Mitigation Strategies

- Rewrite the vulnerable line in **Exercise 1** using strncpy.
- Explain in one sentence how a "Stack Canary" would prevent Exercise 2 from working.